

ALBERI AVL

PIETRO DI LENA

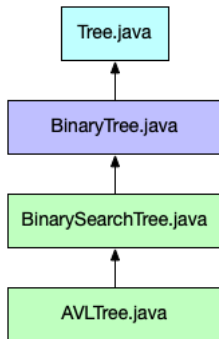
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



ESERCIZIO

- Implementare la classe *AVLTree* estendendo la classe *BinarySearchTree*



- Il metodi *search*, *size*, *isempty* non necessitano di modifiche rispetto a quelli forniti nella classe *BinarySearchTree*
- L'inserimento/rimozione di un nodo viene effettuata come nella classe *BinarySearchTree* ma dopo ogni inserimento/rimozione è necessario riaggiornare le altezze (campo *height*) di ogni nodo ed eventualmente effettuare rotazioni per ribilanciare l'albero

STRUTTURA DELLA CLASSE AVLTree

```
1 public class AVLTreeNode<K extends Comparable<K>,D> extends BinarySearchTree<K,D> {
2     public AVLTree() {
3         super();
4     }
5     @Override
6     public void insert(K key, D data) throws IllegalStateException { ... }
7     @Override
8     public D delete(K key) { ... }
9
10    ...
11
12    protected class AVLNode extends BTreeNode {
13        protected int height;
14
15        public AVLNode(K key, D data) {
16            super(key,data);
17            height = 0;
18        }
19
20        public int balanceFactor() { ... }
21        public void updateHeight() { ... }
22        public void rightRotate() throws IllegalStateException { ... }
23        public void leftRotate() throws IllegalStateException { ... }
24        public void balance() { ... }
25        ...
26    }
27 }
```

- La classe *AVLNode* estende *BTreeNode*, introducendo il campo *height*
- Inoltre, espone una serie di metodi per controllare ed effettuare il ribilanciamento dell'albero, come ad esempio le rotazioni

PSEUDOCODICE DI INSERT

```
1: function INSERT(AVLTree  $T$ , KEY  $k$ , DATA  $d$ )
2:   if  $k \neq \text{NIL}$  then
3:      $v = \text{AVLNode}(k, d)$ 
4:     INSERTNODE( $T, v$ )
5:      $p = v.\text{parent}$ 
6:     while  $p \neq \text{NIL}$  and  $|\text{BALANCEFACTOR}(p)| \neq 2$  do
7:       UPDATEHEIGHT( $p$ )
8:        $p = p.\text{parent}$ 
9:     if  $p \neq \text{NIL}$  then
10:      BALANCE( $T, p$ )
```

- INSERTNODE a riga 4 è il metodo sviluppato per Alberi Binari
- Notare che a riga 4 inseriamo nell'albero binario nodi di tipo *AVLNode*
- BALANCE, BALANCEFACTOR e UPDATEHEIGHT sono metodi disponibili nella classe *AVLNode*, basta invocarli su p
- In Java, a seconda delle scelte implementative, gli assegnamenti a riga 5 e 8 potrebbero richiedere typecasting, oppure il typecasting deve essere effettuato quando si invocano metodi di *AVLNode*

PSEUDOCODICE DI DELETE

```
1: function DELETE(AVLTree  $T$ , KEY  $k$ )
2:   if  $k \neq \text{NIL}$  then
3:      $v = \text{DELETENODE}(T, k)$ 
4:     if  $v \neq \text{NIL}$  then
5:        $p = v.\text{parent}$ 
6:       while  $p \neq \text{NIL}$  do
7:         if  $|\text{BALANCEFACTOR}(p)| == 2$  then
8:            $\text{BALANCE}(T, p)$ 
9:         else
10:           $\text{UPDATEHEIGHT}(p)$ 
11:         $p = p.\text{parent}$ 
12:      return  $v.\text{data}$ 
13:   return NIL
```

- DELETENODE è il metodo sviluppato per Alberi Binari
 - Tale metodo non modifica il campo *parent* del nodo rimosso v
 - L'assegnamento a riga 5 è quindi valido
- Come per la INSERT, è necessario effettuare dei typecasting per convertire lo pseudocodice in Java